



AURA

Agentic Retargeting Framework

LangGraph Implementation Update

Students: Yunbo Wang

Company Liaisons: Joe Tostenrude, Yousef Iskander, Ph.D., Richard Paw

Faculty Advisor: Dr. Farzad Ahmadkhanlou

Date: May 08, 2026



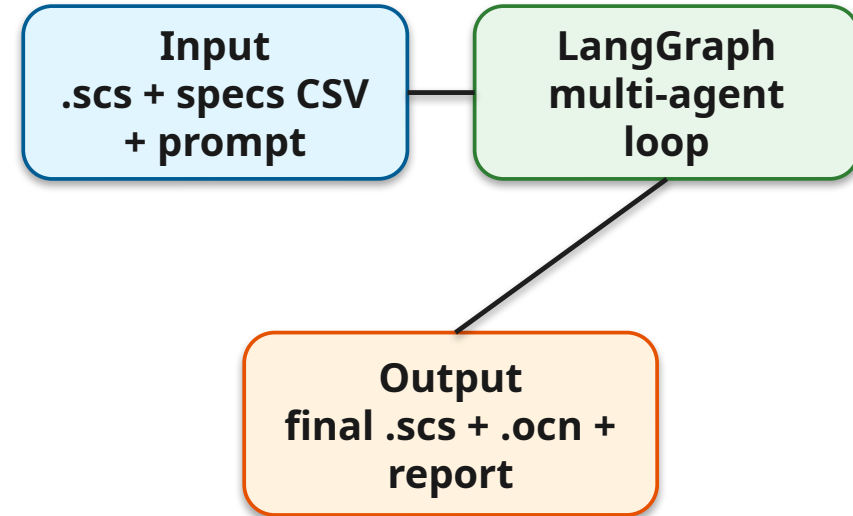
This Deck: Agentic Framework Only

Built the core LangGraph orchestration for analog netlist migration.

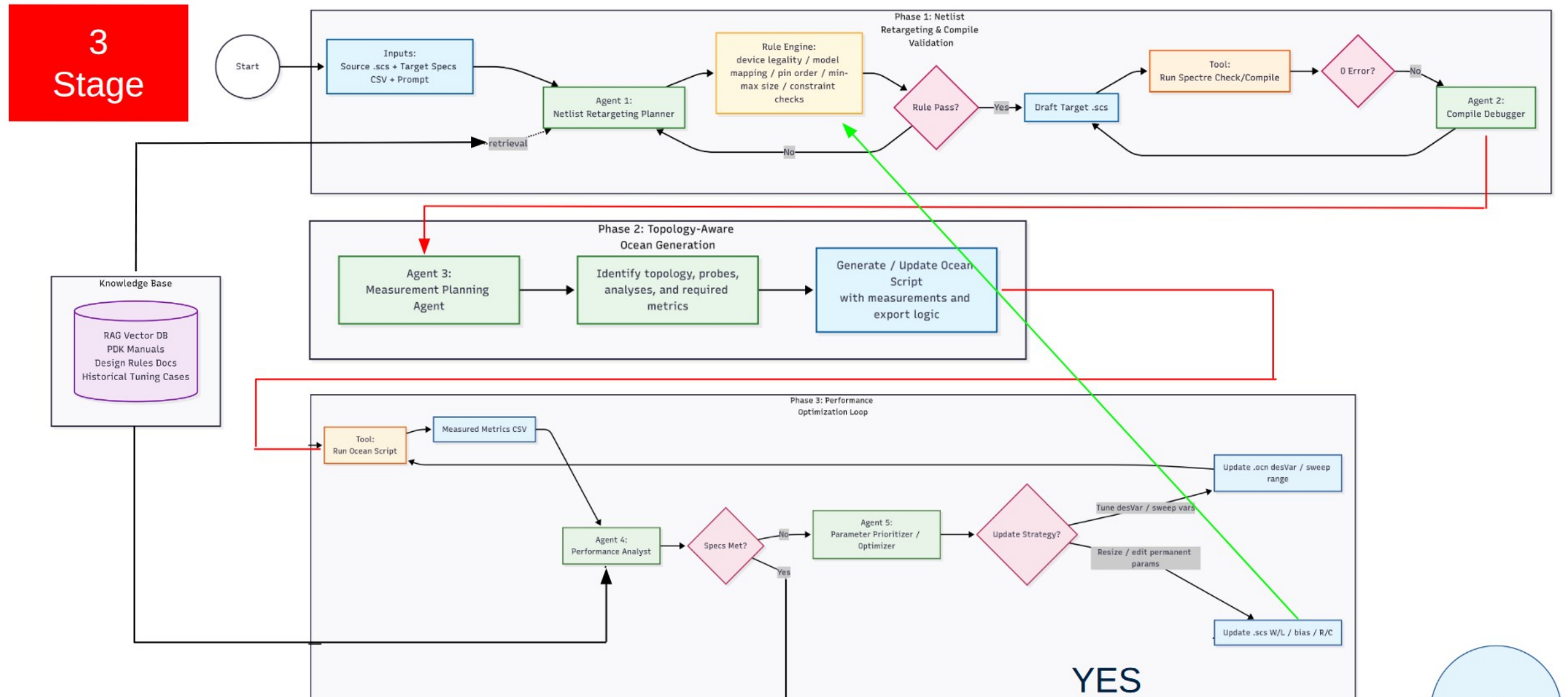
Focus: agent roles, routing logic, Cadence/Ocean tool wrappers, run artifacts.

Not covering model training, GNN decoder, or GLA experiments in this update.

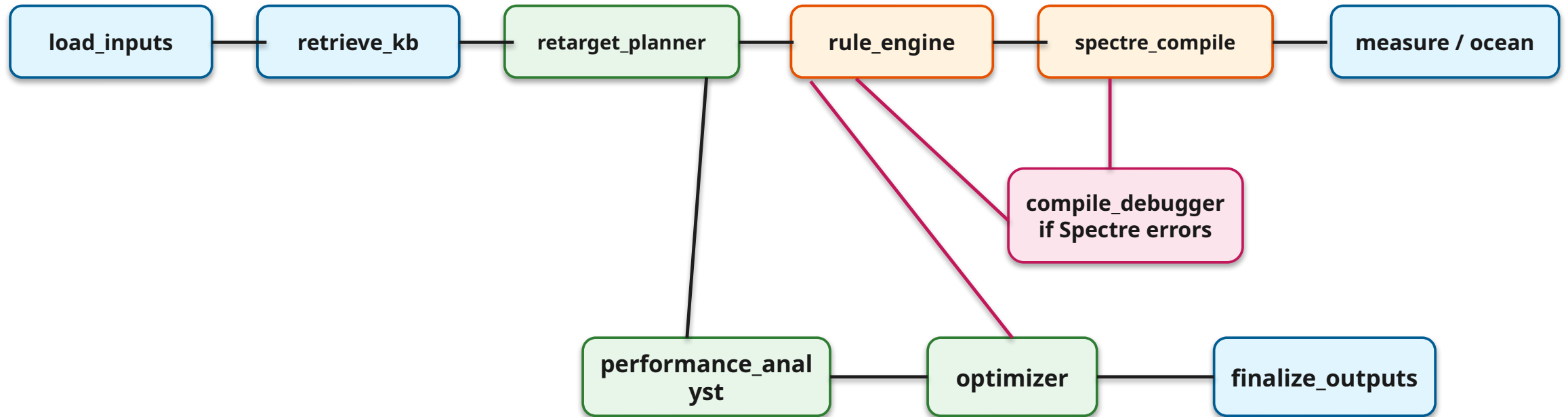
Default mode runs locally without UCI server; Cadence mode is enabled by environment variables.



3 Stage Agent Flow



LangGraph Runtime



Graph Code: Explicit Routing

```
19
20
21 def route_after_rules(state: AuraState) -> str:
22     if state.get("rule_pass", False):
23         return "compile"
24     if state.get("retarget_attempt", 0) < state.get("max_retarget_attempts", 3):
25         return "retarget"
26     return "finalize"
27
28
29 def route_after_compile(state: AuraState) -> str:
30     if state.get("spectre_compile_ok", False):
31         return "measure"
32     if state.get("compile_debug_attempt", 0) < state.get("max_compile_debug_attempts", 3):
33         return "debug"
34     return "finalize"
35
36
37 def route_after_analysis(state: AuraState) -> str:
38     if state.get("specs_met", False):
```

State Contract

Inputs

source_scs_path
target_specs_csv_
path
target_pdk
user_prompt

Retargeting

retarget_plan
draft_scs_path
rule_violations
rule_pass

Tools

spectre_compile_
ok
ocean_script_pat
h
measured_csv_pa
th
compile_errors

Optimization

specs_met
failed_specs
optimization_acti
ons
final_scs_path

The graph passes one TypedDict state across every agent and tool node.
All generated files are also registered in `artifacts` for reporting.

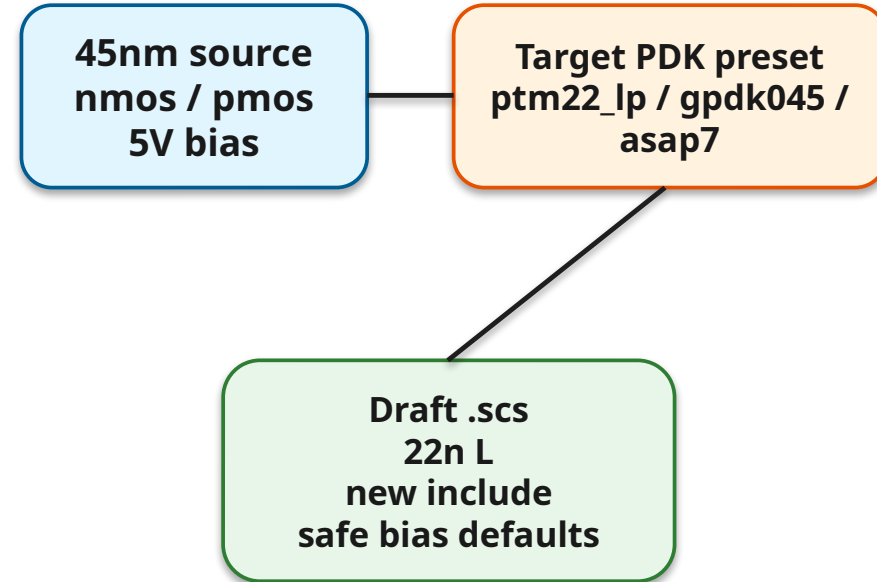
Phase 1: Netlist Retargeting

Reads source Spectre netlist and target specs CSV.

Builds a first-pass retarget plan from local PDK presets and examples.

Rewrites model include, MOS model names, default bias variables, and target L/W constraints.

Rule engine checks model mapping, MOS pin count, min length/width, and required design parameters.



Phase 1 Code: Planner and Rule Gate

```
57     state["max_retarget_attempts"] = settings.max_retarget_attempts
58     state["max_compile_debug_attempts"] = settings.max_compile_debug_attempts
59     state.setdefault("max_iterations", settings.max_iterations)
60     state.setdefault("iteration", 0)
61     state.setdefault("retarget_attempt", 0)
62     state.setdefault("compile_debug_attempt", 0)
63     state.setdefault("optimization_actions", [])
64     state.setdefault("artifacts", {})
65     state.setdefault("events", [])
66
67     copied_source = run_dir / "input_source.scs"
68     write_text(copied_source, source_netlist)
69     _artifact(state, "input_source", copied_source)
70     _artifact(state, "target_specs", specs_path)
71     event(state, f"Loaded source netlist and {len(target_specs)} target specs.")
72     return state
73
74
75 def retrieve_kb(state: AuraState) -> AuraState:
76     query = "\n".join(
```

Tool Node: Spectre Compile / Debugger

`run\_spectre\_compile` prepares the real Cadence command and parses logs.

`AURA\_DRY\_RUN=true` skips external binaries for local development.

When connected to UCI server, set `AURA\_DRY\_RUN=false` and source Cadence profile.

Compile debugger keeps edits conservative: fix missing params, then route back to rule engine.

```
spectre -64 draft_target.scs +escchars +log spectre_compile.log +aps -
```

No SSH inside the agent
Run locally on UCI Linux/Cadence host

Phase 2: Topology-Aware Ocean Generation

Infers differential amplifier probes: Vin+, Vin-, Vout1, Vout2, VDD, ground.

Chooses analyses from requested metrics: dcOp, ac, transient.

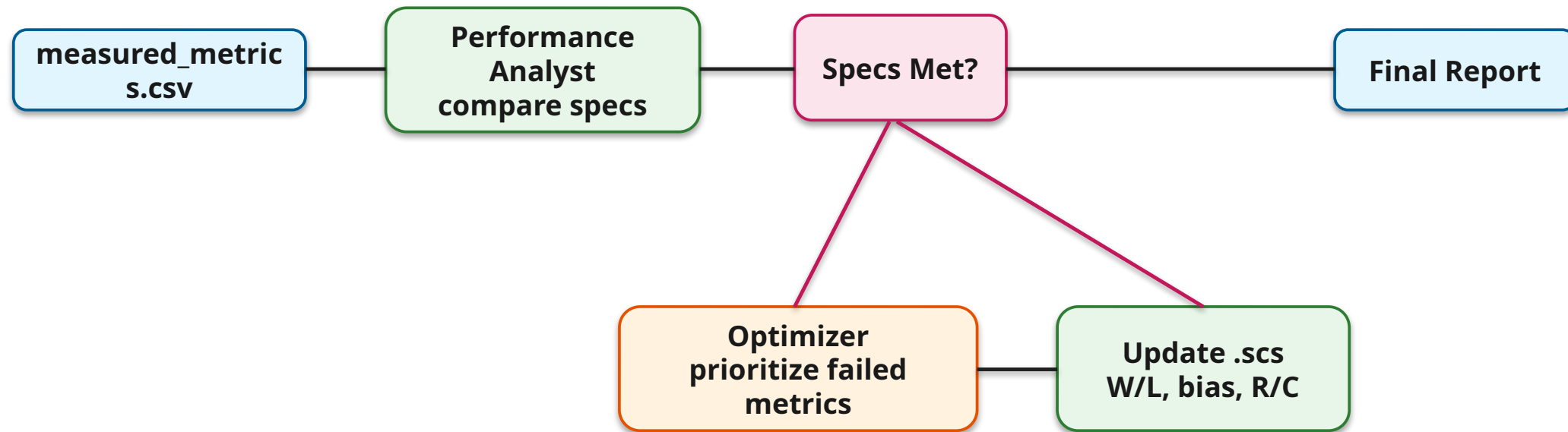
Generates `.ocn` with simulator, modelFile, desVar, analysis, resultsDir, ocnPrint.

Normalizes Spectre values like `10uA`, `0.95V`, `10mV` into Ocean-friendly literals.



```
simulator( 'spectre ' )
modelFile( list( "22nm_LP.pm" "" ) )
desVar( "VDD" 0.95 )
analysis('tran ?stop "5u" )
ocnPrint( v("/Vout2") ?output "waveforms.csv" )
```

Phase 3: Performance Optimization Loop



Gain or bandwidth failures tune R_g/R_m .

Power failures reduce ISS/I_{btail}.

Output common-mode failures tune A_{ref}/V_{b3} .

Tooling Code: Metrics and Optimization

```
389     "parameter_updates": default_params,
390     "notes": [
391         "Topology is preserved.",
392         "Bias defaults come from local David/UCI examples and can be overridden by target specs CSV
393     ],
394 }
395
396
397 def apply_retargget_plan(netlist: str, plan: dict[str, Any]) -> str:
398     include = plan.get("include_update", {})
399     pdk_name = plan.get("target_pdk", "")
400     pdk = get_pdk(pdk_name)
401     text = replace_include_lines(netlist, include.get("path", ""), include.get("suffix", ""))
402     text = replace_parameters_block(text, plan.get("parameter_updates", {}))
403     text = rewrite_mos_devices(
404         text,
405         nmos_model=pdk.get("nmos_model", "nmos"),
406         pmos_model=pdk.get("pmos_model", "pmos"),
407         min_l=plan.get("geometry_updates", {}).get("min_l", ""),
408         min_w=plan.get("geometry_updates", {}).get("min_w", ""),
```

Agent Code Files

```
.env.example  
.gitignore  
README.md  
codex_agent/__init__.py  
codex_agent/config.py  
codex_agent/graph.py  
codex_agent/nodes.py  
codex_agent/state.py  
codex_agent/tools.py  
examples/target_specs_example.csv  
requirements.txt  
run_agent.py  
scripts/generate_agent_ppt.py
```

- graph.py: LangGraph topology and conditional routers.
- nodes.py: agent/tool nodes matching the three-stage flow.
- tools.py: netlist parser, PDK presets, rule checks, runner wrappers.
- state.py: shared state contract.
- run_agent.py: reproducible CLI entrypoint.

How To Run The Agent

```
cd /home/emc/Documents/rag/codex_agent
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

python run_agent.py \
  --source /home/emc/Documents/rag/david/45nm/2stageinput_david.scs \
  --target-specs examples/target_specs_example.csv \
  --target-pdk ptm22_lp \
  --prompt "Retarget the 45nm amplifier to PTM 22nm LP"
```

Outputs:

```
workspace/runs/<run_id>/
outputs/<run_id>/final_target.scs
outputs/<run_id>/measurements.ocn
outputs/<run_id>/migration_report.md
```



Current Technical Features

Local KB Retrieval

Reads david / csc / TPM / mother_code examples for relevant snippets.

PDK Presets

ptm22_lp, gpdk045, asap7 model names, min geometry, include paths.

Dry Run First

Develop and demo without Cadence connection; no SSH required.

Cadence Ready

Spectre/Ocean wrappers use environment variables for UCI server mode.

Artifact Tracking

Every run writes draft .scs, .ocn, logs, metrics CSV, final report.

Loop Routing

Optimizer sends updates back to rule checks or Ocean generation.



Next Steps

Run `AURA\_DRY\_RUN=false` on the UCI Cadence Linux host and validate Spectre/Ocean logs.

Replace heuristic retarget planner with LLM + structured RAG over PDK manuals and tuning cases.

Add stronger analog metric extraction: gain, bandwidth, phase margin, power, settling, swing.

Connect Skillbridge/Virtuoso path for schematic-level desVar tuning.

Later stage: log trajectories for Agent Lightning style policy/router improvement.